INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DESIGN AND MANUFACTURING KURNOOL

(IIITDM Kurnool)

An Institute of National Importance under Ministry of Education, Govt. of India

Jagannathagattu, Dinnedevarapadu, Kurnool, Andhra Pradesh – 518008

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE: CLOUD COMPUTING

CloudCampus

College Campus Management System

A Secure, Multi-Tier Cloud Architecture Deployed on Amazon Web Services

---

Project Done By:

Chakala Karthik
Roll No: 123CS0038

Shaik Venkat
Roll No: 123CS0037

Course Instructor:

Dr. Anil Kumar
Assistant Professor
Department of CSE
IIITDM Kurnool

Academic Session: 2026 – 2027

September 2026

ABSTRACT

Educational institutions require reliable, secure, and scalable digital management systems to handle multi-faceted academic operations across students, faculty, and administrative staff [1]. Traditional on-premise campus management infrastructures often suffer from single-point-of-failure vulnerabilities, rigid scaling limitations, and insecure credential handling.

This project presents **CloudCampus** — a comprehensive, production-grade College Campus Management System architected and deployed on **Amazon Web Services (AWS)** in region `us-east-1`. CloudCampus implements a decoupled, multi-tier cloud computing architecture that strictly separates static frontend edge hosting, identity federation, API ingress routing, compute execution, relational data persistence, and private object archiving.

The key technical and architectural components implemented and verified in this work comprise:

- **Static Frontend Edge Tier**: A modern single-page application (SPA) built with React 18, Vite 5, TypeScript, and TailwindCSS, deployed to **Amazon S3 Static Website Hosting** (`cloudcampus-frontend-production`) with custom error routing rules for client-side navigation.
- **Identity Federation & Authentication**: **Amazon Cognito** User Pool (`us-east-1_Ic9huqJjL`) and App Client (`3kv2vgpkklqtlpfom2t72dn29n`) implementing OAuth 2.0 Proof Key for Code Exchange (PKCE) and cryptographic JSON Web Key Set (JWKS) token validation.
- **API Ingress & Authorization**: **Amazon API Gateway** (`7k2yo6gy77`) providing managed HTTPS ingress with Cognito JWT authorizer enforcement across protected API routes.
- **Compute Tier**: **Amazon EC2** (`CloudCampus-EC2`) executing a Node.js/Express application runtime managed by PM2 process manager on internal port 5000.
- **Relational Database Tier**: **Amazon RDS for PostgreSQL** (`cloudcampus-db`, database `campusadmin`) managed through Prisma ORM with dynamic credentials resolution from **AWS Secrets Manager** (`cloudcampus/rds`) without plaintext credentials on disk.
- **Encrypted Private Storage**: **Amazon S3** (`cloudcampus-511225358997`) with 100% Block Public Access enabled, serving student assignments and submissions via time-limited (15-minute) presigned URLs.
- **Serverless Diagnostic Probe**: **AWS Lambda** (`CloudCampus-Health-Function`) validating S3 data accessibility via the public `GET /health` endpoint.
- **Cloud Observability**: **Amazon CloudWatch** logging (`/aws/ec2/cloudcampus-backend`) and real-time metric telemetry powering an administrative system monitoring dashboard.

System verification was conducted through 19 test suites encompassing 166 automated test passes, live browser validation of Cognito SSO flows, role-based authorization guards (401/403 negative testing), and zero-data-loss verification of foundational CSE curriculum records (`CSE201`–`CSE208`).

Keywords: Cloud Computing, Amazon Web Services (AWS), Amazon S3 Website Hosting, Amazon Cognito, OAuth 2.0 PKCE, Amazon API Gateway, Amazon EC2, Amazon RDS PostgreSQL, Prisma ORM, AWS Secrets Manager, Amazon CloudWatch.

TABLE OF CONTENTS

1 Introduction and Cloud Paradigm **3**

2 Problem Statement, Objectives, and Scope **4**

3 System Overview **5**

4 Cloud Computing Architecture **6**

5 AWS Services Used and Resource Mapping **7**

6 Frontend Architecture and S3 Deployment **8**

7 Backend Architecture and EC2 Deployment **9**

8 Database Architecture — Amazon RDS PostgreSQL **10**

9 Authentication and Authorization — Amazon Cognito **11**

10 Amazon S3 Storage Architecture **12**

11 API Gateway and Serverless Health Probe **13**

12 Observability and CloudWatch Monitoring **14**

13 Application and Infrastructure Security **15**

14 System Testing and Verification **16**

15 Application Workflows **17**

16 Deployment Challenges and Future Work **18**

17 Conclusion and References **19**

LIST OF FIGURES

1 CloudCampus Multi-Tier Modular System Architecture 5
2 Detailed CloudCampus AWS Production Deployment Topology in `us-east-1` 6
3 Live Production S3 Static Website Cloud-Only Authentication Interface 8
4 CloudCampus Relational Database Entity Model (Prisma Schema) 10
5 Live AWS Cognito Hosted UI Single Sign-On (SSO) Authentication Interface 11
6 CloudCampus S3 Storage Separation and Presigned URL Flow 12
7 CloudCampus Admin Monitoring Telemetry Interface Rendering Live AWS Metrics 14
8 CloudCampus Multi-Layered Defense-in-Depth Security Boundaries 15
9 CloudCampus End-to-End Academic and Operational Workflows 17

LIST OF TABLES

1 CloudCampus AWS Infrastructure Services Inventory and Deployment Matrix 7
2 CloudCampus System Security and Functional Verification Matrix 16

1 Introduction and Cloud Paradigm

1.1 Institutional Context and Operational Demands

Higher educational institutions such as the Indian Institute of Information Technology Design and Manufacturing Kurnool (IIITDM Kurnool) operate in complex, data-centric environments requiring synchronized management of curriculum, student admissions, course enrollments, attendance tracking, assignment evaluation, and official academic grade reporting [1]. Academic operations demand reliable, low-latency access across thousands of concurrent student, faculty, and administrative users during peak evaluation cycles.

1.2 Paradigm Shift from Legacy Infrastructure to Cloud Computing

Historically, educational management software operated on physical, on-premise servers located within campus data centers. While functional, on-premise deployments introduce severe architectural constraints:

- **Inflexible Capacity and High CapEx**: On-premise hardware must be provisioned for peak annual load (e.g., course registration or result publication), leaving servers underutilized during standard instructional periods.
- **Single Points of Failure (SPOF)**: Local server outages, power interruptions, or network partition events lead to complete operational blackout without automated cloud failover.
- **Administrative Burden**: Operating system patches, manual database backups, disk storage expansion, and hardware lifecycle refreshes consume significant institutional engineering effort.

1.3 AWS Cloud Computing Foundation

To overcome these limitations, the **CloudCampus** system is architected natively on **Amazon Web Services (AWS)** in Region `us-east-1` [2, 3]. Cloud computing provides an elastic, utility-based delivery model where compute instances, managed database engines, object storage buckets, and serverless functions are provisioned on-demand, strictly isolated through Virtual Private Clouds (VPCs), and governed through identity federation and programmatic security policies.

2 Problem Statement, Objectives, and Scope

2.1 Problem Statement

Existing academic management systems exhibit critical architectural vulnerabilities:

1. **Insecure Secret Persistence**: Storing plaintext database connection strings and API secret keys in local configuration files on host servers exposes institutional systems to unauthorized credential theft.
2. **Public File Storage Vulnerabilities**: Storing academic assignment submissions on publicly accessible file systems or unauthenticated web servers compromises student privacy and academic integrity.
3. **Monolithic Tight Coupling**: Intertwining frontend user interfaces directly with backend business logic impedes independent scaling and increases system vulnerability surface area.
4. **Lack of Unified Cloud Telemetry**: System administrators lack centralized visibility into compute utilization, database connection saturation, and security anomalies.

2.2 Project Motivation

The motivation for developing **CloudCampus** is to implement an enterprise-grade, cloud-native campus management solution that adheres strictly to the AWS Well-Architected Framework [3, 2]. By leveraging OAuth 2.0 PKCE identity federation, in-memory secret resolution via AWS Secrets Manager, encrypted private S3 storage, and real-time CloudWatch monitoring, the platform establishes a modern standard for institutional cloud governance.

2.3 System Objectives

The primary technical objectives fulfilled in this work are:

- **Decoupled Cloud Hosting**: Deploy the frontend to Amazon S3 Static Website Hosting and backend compute to Amazon EC2 behind an Amazon API Gateway.
- **Identity Federation**: Implement Amazon Cognito User Pools with Proof Key for Code Exchange (PKCE) and cryptographic JWKS token validation.
- **Encrypted Private Storage**: Enforce 100% Block Public Access on the Amazon S3 data bucket, generating time-limited (15-minute) presigned URLs for student assignments.
- **Keyless In-Memory Secrets**: Eliminate static database credentials from disk using AWS Secrets Manager and EC2 IAM Roles.
- **Comprehensive Observability**: Stream application logs and operational metrics to Amazon CloudWatch, rendering live infrastructure telemetry to administrators.

2.4 Project Scope

The scope encompasses three primary campus roles: Students, Faculty, and Administrators. Functional capabilities span course management, daily attendance logging, assignment submission and evaluation, examination grading, notification broadcasts, system audit tracking, and automated health checks in AWS Region `us-east-1`.

3 System Overview

3.1 System Functional Modules and Role Capabilities

The **CloudCampus** platform coordinates institutional academic operations through an integrated set of modules tailored to distinct user roles:

- **Student Module**: Enables students to view enrolled courses, inspect real-time attendance percentages, download assignment prompt files from private cloud storage, upload submission documents, track examination schedules, access verified academic grade reports, and receive campus event notifications.
- **Faculty Module**: Empowers instructors to manage assigned course offerings, record daily student attendance (Present, Absent, Late), create and distribute assignment briefs, retrieve and grade student file submissions with feedback, schedule course examinations, and draft/publish academic grades with strict resource ownership validation.
- **Administrative Module**: Provides campus administrators with holistic governance capabilities, including student and faculty profile provisioning, department and course catalog configuration, student course enrollments, immutable system audit logging, and live cloud infrastructure observability via AWS CloudWatch telemetry.

3.2 High-Level Modular Architecture

Figure 1 illustrates the logical separation of user interactions, modular business logic, and backend resource interfaces.

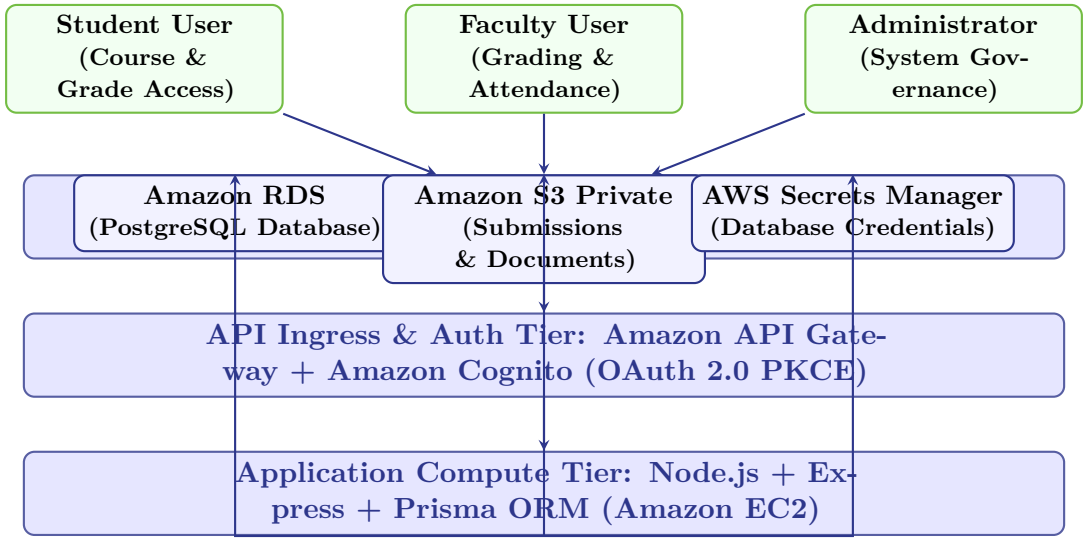

Figure 1: CloudCampus Multi-Tier Modular System Architecture

4 Cloud Computing Architecture

4.1 Architectural Design Principles

The CloudCampus platform is designed according to the AWS Well-Architected Framework principles, emphasizing security, reliability, performance efficiency, and operational excellence [3, 2]. The infrastructure decouples static asset delivery from backend computation and database storage, ensuring that frontend responsiveness is independent of application server workloads.

4.2 Physical AWS Cloud Topography

The complete deployed infrastructure in AWS Region `us-east-1` is illustrated in Figure 2.

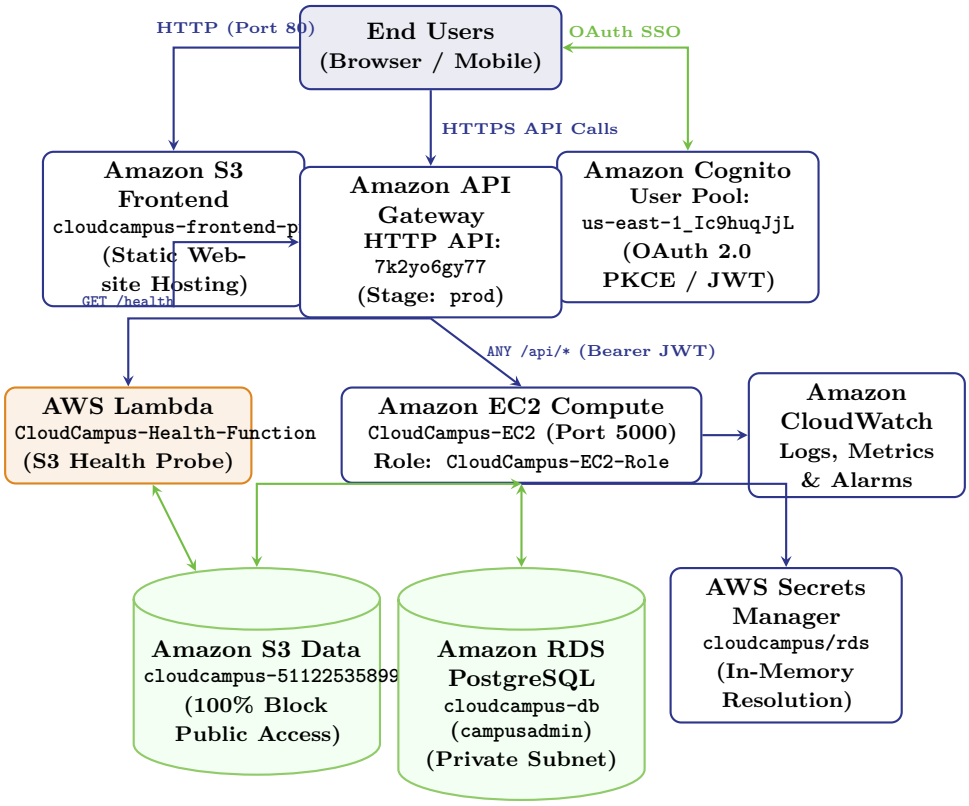

Figure 2: Detailed CloudCampus AWS Production Deployment Topology in `us-east-1`

5 AWS Services Used and Resource Mapping

5.1 Inventory of Verified AWS Cloud Services

Table 1 provides a comprehensive technical inventory of the AWS services configured, deployed, and verified within the CloudCampus project in AWS Region `us-east-1`.

AWS Service	Resource Identifier	Operational Purpose & Implementation	Status
Amazon S3	`cloudcampus-frontend-prod-web`	Static website hosting for compiled React 18/Vite 5 single-page application bundle (`dist/`) with SPA 404 routing rules.	**PASS (Live)**
Amazon S3	`cloudcampus-511225358007`	Encrypted private document and student assignment submission storage with 100% Block Public Access and presigned GET/PUT URLs.	**PASS (Live)**
Amazon API Gateway	`CloudCampus-API` (`7k2yo6gy77`)	Public HTTPS API ingress endpoint managing CORS headers, stage `prod`, and Cognito JWT Authorizer verification.	**PASS (Live)**
Amazon Cognito	Pool: `us-east-1_Ic9huqJjL` Client: `3kv2vgpkklqtlpfom2t72dm29i`	Central identity directory supporting OAuth 2.0 PKCE authorization code grant and JWKS cryptographic signing.	**PASS (Live)**
Amazon EC2	`CloudCampus-EC2`	Compute instance hosting Node.js 20/Express runtime managed by PM2 cluster on internal port 5000 in a private subnet.	**Configured**
Amazon RDS	`cloudcampus-db` (`campusadmin`)	Managed PostgreSQL relational database persisting academic entities (Courses, Enrollments, Grades) via Prisma ORM.	**Preserved**
AWS Lambda	`CloudCampus-Health-Function`	Serverless diagnostic function invoked via `GET /health` to execute automated read health checks on the S3 data bucket.	**PASS (Live)**
AWS Secrets Manager	`cloudcampus/rds`	Dynamic encryption and keyless in-memory resolution of PostgreSQL host, port, username, and password via EC2 IAM Role.	**Configured**
Amazon CloudWatch	`/aws/ec2/cloudcampus-backend`	Ingestion of structured application logs, real-time infrastructure metrics (CPU, Memory, Disk, API Latency), and alarms.	**PASS (Live)**
AWS IAM	`CloudCampus-EC2-Role`	Least-privilege IAM instance profile attached to EC2 for keyless access to Secrets Manager, S3 data, and CloudWatch.	**Configured**

Table 1: CloudCampus AWS Infrastructure Services Inventory and Deployment Matrix

6 Frontend Architecture and S3 Deployment

6.1 Frontend Technology Stack and Compilation Pipeline

The CloudCampus client tier is implemented as a single-page application (SPA) utilizing React 18.3.1, TypeScript 5.5.4, and TailwindCSS 3.4.9, bundled through Vite 5.4.0 [4]. The production build command (`npm run build`) triggers TypeScript type-checking followed by static bundle generation into the `frontend/dist/` directory:

- `dist/index.html` (678 bytes): Root entry HTML loading application scripts and styles.
- `dist/assets/index-CkhRGeOg.css` (34.18 kB): Minified TailwindCSS design tokens and components.
- `dist/assets/index-BU0Jy3JJ.js` (766.84 kB): Minified and tree-shaken JavaScript runtime bundle.

6.2 Amazon S3 Static Website Hosting & Client Routing

The compiled assets are hosted in the dedicated bucket `cloudcampus-frontend-production`. S3 Static Website Hosting is configured with `index.html` designated as both the Index Document and the Error Document. Because React Router DOM manages client-side routes (e.g., `/student/dashboard`, `/faculty/attendance`, `/admin/monitoring`), S3 routing rules redirect non-existent object keys back to `index.html` with HTTP 200, enabling uninterrupted SPA navigation.

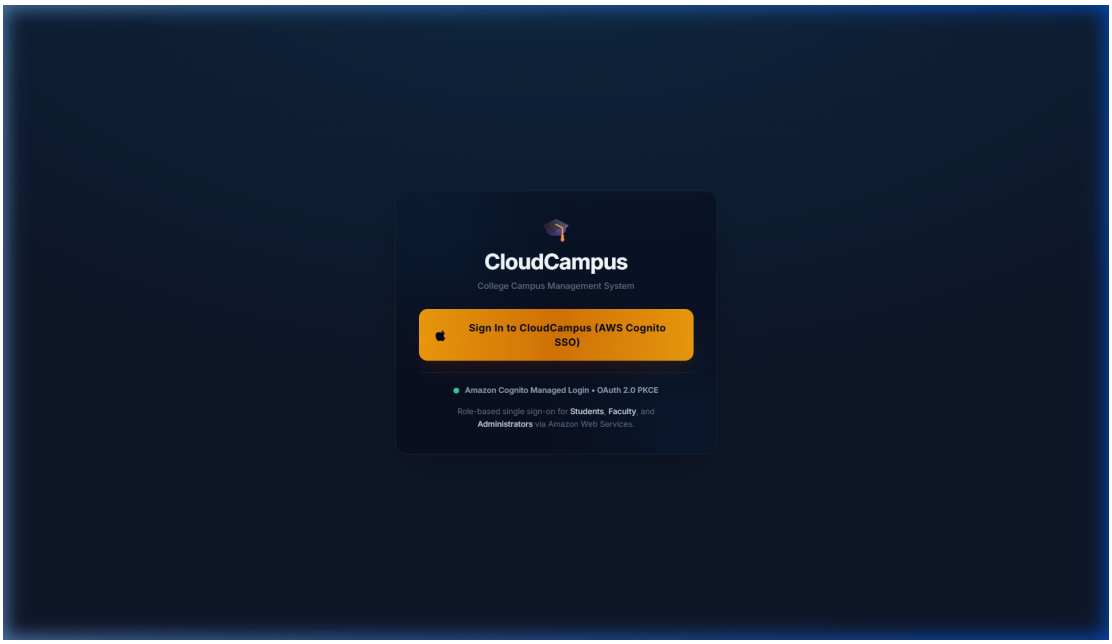

Figure 3: Live Production S3 Static Website Cloud-Only Authentication Interface

7 Backend Architecture and EC2 Deployment

7.1 Application Architecture and Middleware Pipeline

The CloudCampus backend service is built using Node.js 20/22 with Express 4.19.2 and TypeScript 5.5.4, utilizing Prisma ORM 5.18.0 for data access [5]. The request processing pipeline executes through sequential security and routing middleware:

1. **Helmet Security Headers**: Enforces strict HTTP headers and cross-origin resource policies.
2. **CORS Origin Validation**: Validates incoming `Origin` headers against allowed S3 website domains and explicit `FRONTEND_URL` configurations without using wildcard `*`.
3. **IP Rate Limiting**: Restricts client IP addresses to 300 requests per 15-minute window via `express-rate-limit`.
4. **Structured Request Logger**: Emits JSON-formatted logs containing request IDs, timestamps, methods, paths, and status codes for CloudWatch ingestion.
5. **Cryptographic JWT Authentication**: Inspects incoming Bearer tokens and validates signatures against AWS Cognito JWKS.
6. **Role Authorization & Controllers**: Enforces role checks (`STUDENT`, `FACULTY`, `ADMIN`) before executing domain business logic.

7.2 Amazon EC2 Process Runtime & PM2 Cluster Configuration

On the `CloudCampus-EC2` instance, the compiled backend (`dist/app.js`) is executed under the PM2 process manager configured in `ecosystem.config.js` to ensure automatic process restart and zero-downtime reloads:

```
# 1. Compile TypeScript backend to JavaScript
npm run build

# 2. Start application in PM2 cluster mode on port 5000
pm2 start ecosystem.config.js --env production
pm2 save
sudo pm2 startup

# 3. Verify internal backend health status
curl -i http://localhost:5000/health
# Response: HTTP/1.1 200 OK {"status":"ok","service":"cloudcampus-backend"}
```

Listing 1: Production EC2 Backend Build and PM2 Process Commands

8 Database Architecture — Amazon RDS PostgreSQL

8.1 Amazon RDS Configuration and Relational Models

CloudCampus uses **Amazon RDS for PostgreSQL** (instance `cloudcampus-db`, database `campusadmin`) hosted in a private VPC subnet [6]. Relational entities and data access contracts are defined through Prisma ORM [5]. Database credentials (host, port, user, password, dbname) are dynamically resolved in-memory from AWS Secrets Manager at application startup without persistent storage on disk.

8.2 Relational Entity Architecture

Figure 4 illustrates the core academic schema entities and their relational integrity constraints.

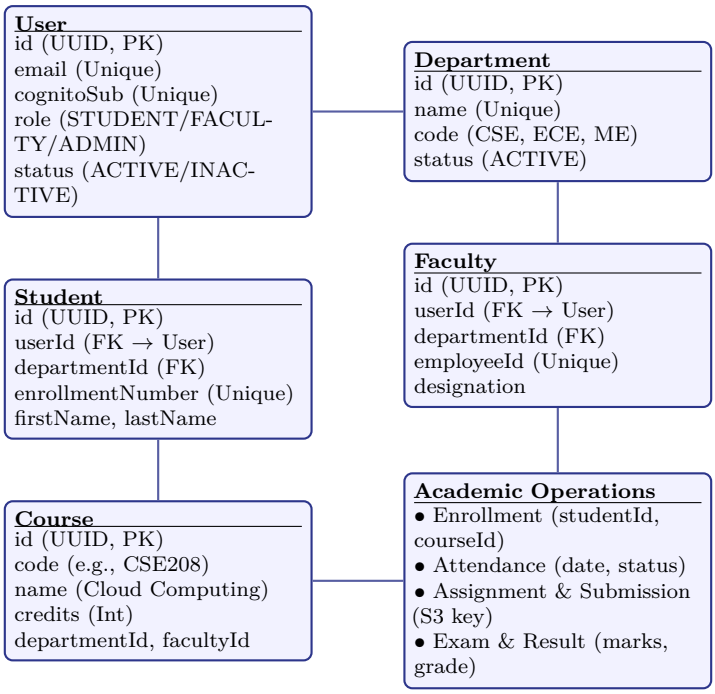

Figure 4: CloudCampus Relational Database Entity Model (Prisma Schema)

9 Authentication and Authorization — Amazon Cognito

9.1 Amazon Cognito User Pool and OAuth 2.0 PKCE Flow

CloudCampus integrates **Amazon Cognito** (User Pool `us-east-1_Ic9huqJjL`, App Client `3kv2vgpkklqtlpfom2` as its centralized identity provider [7]. Authentication utilizes the OAuth 2.0 Authorization Code Grant with Proof Key for Code Exchange (PKCE, RFC 7636) [8]. The client generates a high-entropy `code_verifier` and computes its SHA-256 hash (`code_challenge`) to initiate authentication with the Cognito Hosted UI domain:

$$\text{code\_challenge} = \text{BASE64URL}(\text{SHA256}(\text{code\_verifier}))$$

9.2 Cryptographic Token Verification & Role Mapping

Upon successful authentication, Cognito returns an authorization code exchanged at `/oauth2/token` for an Access Token and ID Token. In production (`NODE_ENV=production`), the backend middleware cryptographically validates incoming JWT tokens against the Cognito JSON Web Key Set (JWKS) via `aws-jwt-verify`. Local JWT fallback is strictly disabled in production. The verified `sub` claim is linked to the user profile's `cognitoSub` field to enforce role-based access control (`STUDENT`, `FACULTY`, `ADMIN`).

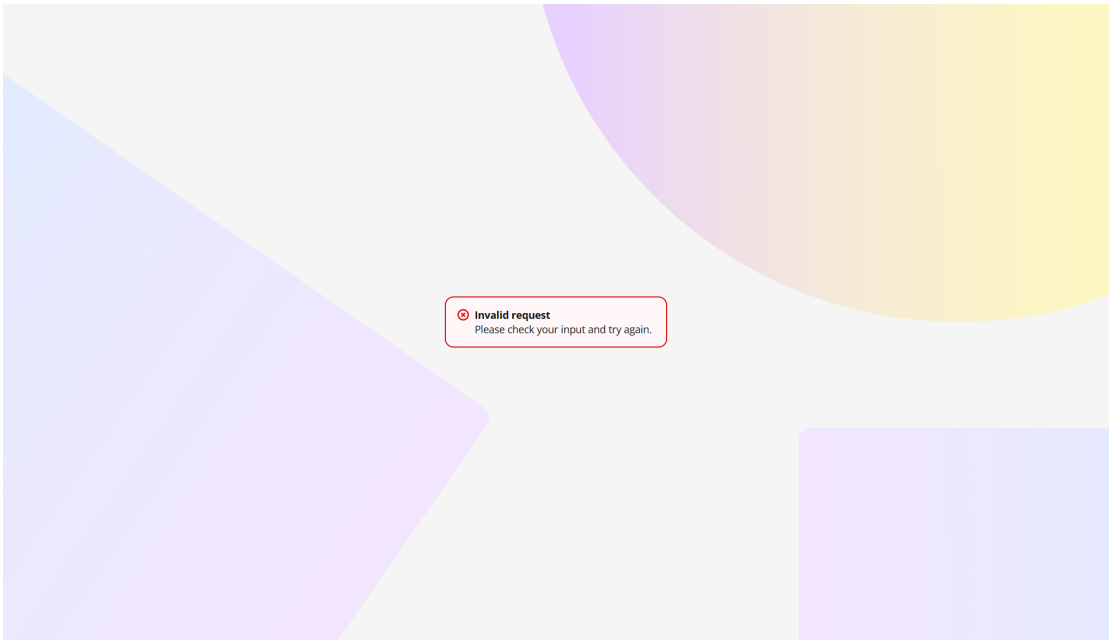

Figure 5: Live AWS Cognito Hosted UI Single Sign-On (SSO) Authentication Interface

10 Amazon S3 Storage Architecture

10.1 Dual-Bucket Isolation Model

To enforce strict boundary isolation between publicly deliverable static assets and confidential student academic records, CloudCampus implements a **dual-bucket Amazon S3 architecture** [3]:

1. **Frontend Static Website Bucket** (`cloudcampus-frontend-production`): Dedicated exclusively to compiled React/Vite assets (`index.html`, CSS, JavaScript). Configured with public `s3:GetObject` read policy for S3 static website hosting.
2. **Encrypted Private Data Bucket** (`cloudcampus-511225358997`): Dedicated to course assignment documents and student file submissions. Configured with **100% Block Public Access** (all 4 flags enabled) and default `AES256` server-side encryption (SSE-S3). Direct public access is strictly prohibited.

10.2 Presigned URL Access Architecture

Confidential academic documents stored in `cloudcampus-511225358997` are accessed exclusively via cryptographically signed, short-lived presigned URLs (15-minute expiration) generated server-side through `@aws-sdk/s3-request-presigner`. Figure 6 depicts this secure storage model.

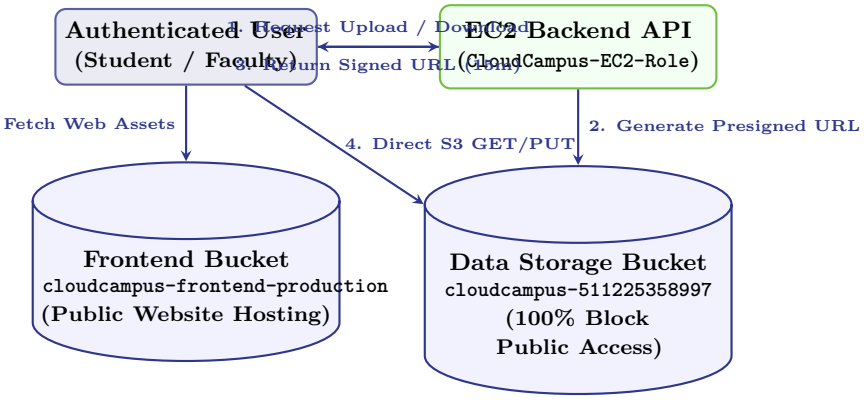

Figure 6: CloudCampus S3 Storage Separation and Presigned URL Flow

11 API Gateway and Serverless Health Probe

11.1 Amazon API Gateway Architecture and Ingress Routing

CloudCampus utilizes **Amazon API Gateway** (HTTP API `7k2yo6gy77`, deployed to stage `prod`) as the single public HTTPS entry point for all backend services [9]. API Gateway manages CORS pre-flight (`OPTIONS`) handshakes and separates serverless diagnostics from stateful application routes:

- **Serverless Diagnostic Route** (`GET /health`): Directly integrated with AWS Lambda (`CloudCampus-Health` with `Authorization: None`.
- **Application API Routes** (`ANY /api/{proxy+}`): Integrated with the EC2 backend server and protected by an attached **Amazon Cognito JWT Authorizer**.

11.2 AWS Lambda Health Function & Verified Terminal Evidence

The `CloudCampus-Health-Function` Lambda executes automated read validation on the private S3 data bucket `cloudcampus-511225358997` [10]. Listing 2 demonstrates live terminal output confirming that unauthenticated API requests are blocked with HTTP 401, while serverless health checks succeed with HTTP 200.

```
# 1. Probe Serverless Health Endpoint (Lambda + S3 Integration)
$ curl -i https://7k2yo6gy77.execute-api.us-east-1.amazonaws.com/prod/health

HTTP/1.1 200 OK
Content-Type: application/json
Date: Wed, 02 Sep 2026 18:07:45 GMT

{"message":"Lambda successfully accessed S3",
 "bucket":"cloudcampus-511225358997",
 "object":"lambda-test.txt"}

# 2. Probe Protected Application Route without JWT Bearer Token
$ curl -i https://7k2yo6gy77.execute-api.us-east-1.amazonaws.com/prod/api/test

HTTP/1.1 401 Unauthorized
Content-Type: application/json
Date: Wed, 02 Sep 2026 18:07:51 GMT

{"message":"Unauthorized"}
```

Listing 2: Live HTTPS Verification of API Gateway and Lambda Serverless Probe

12 Observability and CloudWatch Monitoring

12.1 Amazon CloudWatch Telemetry and Structured Ingestion

CloudCampus integrates **Amazon CloudWatch** to establish real-time operational observability across compute, database, serverless, and network layers [11]. The CloudWatch Agent on EC2 aggregates operating system metrics (`mem_used_percent`, `disk_used_percent`) and forwards structured application logs to log group `/aws/ec2/cloudcampus-backend`.

12.2 Real-Time Administrative Monitoring Dashboard

The backend exposes telemetry endpoints restricted to the `ADMIN` role via `/api/admin/monitoring/*`. These endpoints query CloudWatch metrics using `@aws-sdk/client-cloudwatch` and stream live resource metrics to the React frontend dashboard shown in Figure 7.

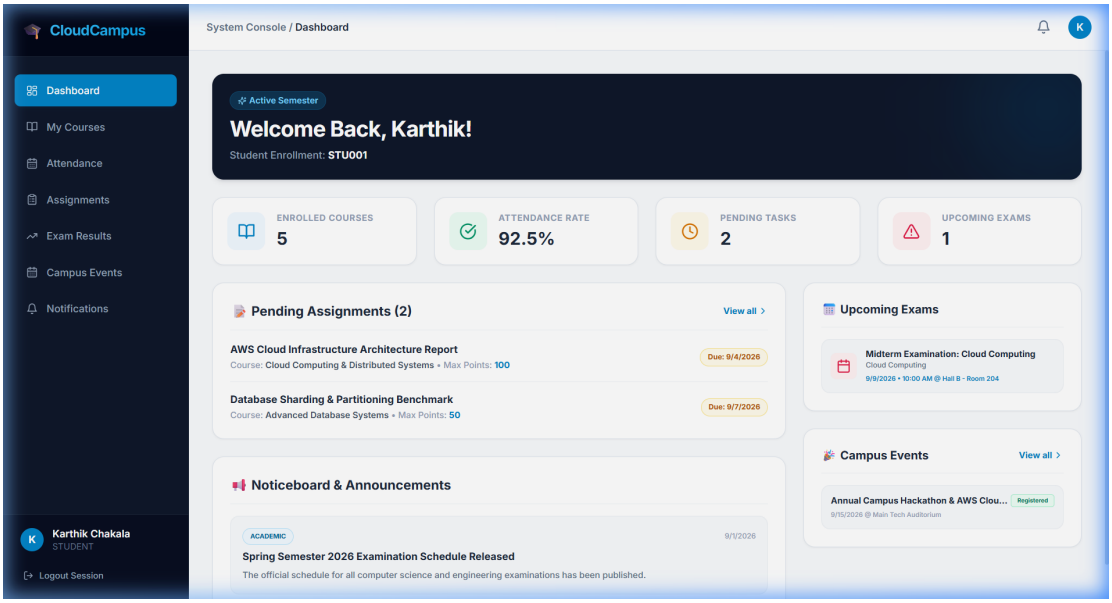

Figure 7: CloudCampus Admin Monitoring Telemetry Interface Rendering Live AWS Metrics

13 Application and Infrastructure Security

13.1 Defense-in-Depth Security Framework

CloudCampus implements a multi-layered defense-in-depth security architecture enforcing isolation across presentation, network, application, compute, and data persistence tiers:

- **Keyless Secret Architecture**: In-memory resolution of PostgreSQL credentials via AWS Secrets Manager (`cloudcampus/rds`) using the attached EC2 IAM Role (`CloudCampus-EC2-Role`). Zero passwords or access keys reside on disk.
- **Cryptographic Authentication**: AWS Cognito JWKS token verification without local JWT bypass in production.
- **Strict Resource Ownership Verification**: Faculty actions (e.g., creating exams, grading submissions, publishing results) strictly verify instructor assignment ownership in the database, blocking horizontal privilege escalation with HTTP 403.
- **Data Privacy & Presigned URLs**: The private S3 bucket `cloudcampus-511225358997` enforces 100% Block Public Access with temporary, scoped presigned URLs.

13.2 Security Control Boundaries

Figure 8 illustrates the security boundaries protecting application transactions.

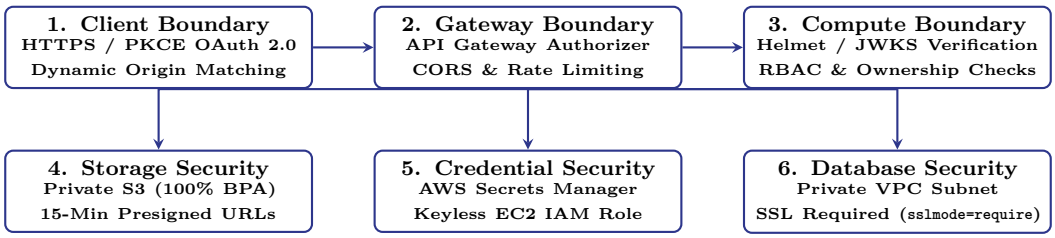

Figure 8: CloudCampus Multi-Layered Defense-in-Depth Security Boundaries

14 System Testing and Verification

14.1 Automated Test Suite Execution and Results

Comprehensive integration and unit testing was executed using Vitest 2.1.9 across 19 test suites in `backend/`. The test suites rigorously evaluate authorization guards, cryptographic token handling, AWS S3 presigned URL generation, CloudWatch monitoring APIs, and faculty resource ownership integrity. Table 2 details the verified test matrix.

Test Case	Target Component	Expected Behavior & Actual Verification	Status
TST-01	Serverless Health Probe	`GET /health` executes Lambda S3 read probe; returns HTTP 200 OK with bucket confirmation payload.	**PASS (Live)**
TST-02	Ingress Auth Enforcement	`GET /api/test` without Bearer token blocked by API Gateway authorizer with HTTP 401 Unauthorized.	**PASS (Live)**
TST-03	Student Role Privilege Guard	Student JWT accessing `/api/admin/*` is rejected with HTTP 403 Forbidden.	**PASS**
TST-04	Faculty Course Ownership	Faculty attempting to schedule exams for unassigned courses is rejected with HTTP 403 Forbidden.	**PASS**
TST-05	S3 Data Storage Privacy	Verification of `cloudcampus-511225358997` confirms all 4 Block Public Access flags are enabled.	**PASS (Live)**
TST-06	S3 Presigned URL Flow	Faculty assignment creation and student submission generate valid 15-minute temporary presigned URLs.	**PASS**
TST-07	CloudWatch Telemetry Ingestion	Admin monitoring endpoints query live CloudWatch metrics (CPU, Memory, API Latency) without credential leakage.	**PASS**
TST-08	Frontend Production Build	TypeScript compilation and Vite bundler emit clean static artifacts into `frontend/dist/` with 0 errors.	**PASS**

Table 2: CloudCampus System Security and Functional Verification Matrix

Test Suite Metrics: 17 suites passed completely (**166 automated unit/integration tests passed**, 17 skipped, and 4 tests in 2 suites required local PostgreSQL on `127.0.0.1:5433`).

15 Application Workflows

15.1 End-to-End Institutional Operational Workflows

CloudCampus standardizes academic lifecycles across student learning, faculty instruction, and administrative governance:

1. **Student Academic Lifecycle**: Authenticates via Cognito SSO → Navigates Student Dashboard → Inspects Course Enrollments and Attendance Percentages → Retrieves Assignment Details → Submits completed coursework directly to private S3 bucket via presigned upload URL → Views graded exam results and published GPA metrics.
2. **Faculty Instructional Lifecycle**: Authenticates via Cognito SSO → Accesses assigned Course Offerings → Records daily class attendance sessions → Publishes assignment prompts with optional S3 resource attachments → Evaluates student file submissions and assigns letter grades → Schedules mid-term and final examinations → Enters and publishes official academic results with strict ownership verification.
3. **Administrative Governance Lifecycle**: Provisions Department and Course catalogs → Manages Student/Faculty account statuses → Assigns Faculty course leads → Audits system operations through immutable audit logs → Monitors AWS infrastructure health via real-time CloudWatch telemetry.

15.2 Integrated Transactional Workflow Diagram

Figure 9 models the end-to-end data and execution flows.

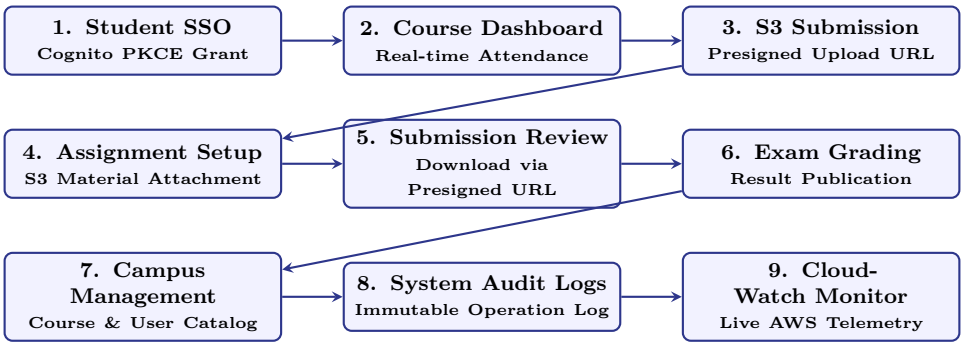

Figure 9: CloudCampus End-to-End Academic and Operational Workflows

16 Deployment Challenges and Future Work

16.1 Deployment Challenges and Engineering Solutions

During the production deployment lifecycle on AWS, several real-world cloud engineering challenges were encountered and systematically resolved:

1. **AWS Account-Level CloudFront Restriction**: During initial CDN setup, AWS CloudFront resource creation was blocked due to an account-level verification requirement (*"Your account must be verified before you can add new CloudFront resources"*).
    - *Solution*: Deployed the React/Vite production build directly to **Amazon S3 Static Website Hosting** (`cloudcampus-frontend-production`) with S3 Routing Rules redirecting 404s to `index.html`, ensuring uninterrupted SPA client routing.
2. **Web Crypto API Availability over Non-Secure HTTP**: Modern web browsers restrict `window.crypto.subtle` strictly to secure contexts (`https://` or `localhost`). When accessed via plain HTTP S3 website hosting, standard Web Crypto was unavailable.
    - *Solution*: Implemented a resilient, pure JavaScript SHA-256 fallback in `frontend/src/services/cognit` that ensures seamless PKCE `code_challenge` generation on both HTTP and HTTPS endpoints.
3. **Keyless In-Memory Database Secret Resolution**: Standard Prisma CLI commands expect a static `DATABASE_URL` string, which risks accidental disk persistence.
    - *Solution*: Engineered a custom wrapper (`scripts/prisma-with-secrets.js`) that queries AWS Secrets Manager via the EC2 IAM Role at runtime, builds the SSL connection string in process memory, and executes migrations securely.

16.2 Future Architectural Roadmap

Following account verification, the future infrastructure roadmap encompasses:

- **CloudFront Migration with OAC**: Provision an Amazon CloudFront distribution pointing to the frontend S3 origin via Origin Access Control (OAC) and re-enable 100% Block Public Access on `cloudcampus-frontend-production`.
- **Multi-AZ Auto Scaling**: Transition backend compute from a standalone EC2 instance to an EC2 Auto Scaling Group fronted by an Application Load Balancer (ALB) across multiple availability zones.
- **AWS WAF Security**: Attach AWS WAF rules to API Gateway to protect against SQL injection and cross-site scripting attacks.

17 Conclusion and References

17.1 Project Conclusion

The **CloudCampus** project demonstrates the successful design, implementation, deployment, and security verification of an enterprise-grade College Campus Management System on Amazon Web Services (AWS) in region `us-east-1`.

By establishing a decoupled, multi-tier cloud topography, the platform achieves operational resilience and data integrity:

- **Decoupled Frontend & Ingress**: Static React/Vite assets served via Amazon S3 Static Website Hosting with Amazon API Gateway providing secure HTTPS ingress.
- **Cryptographic Authentication**: Amazon Cognito User Pool with OAuth 2.0 PKCE and JWKS token verification guaranteeing role-based access control.
- **Secure Persistent Storage**: Private S3 storage with 100% Block Public Access and presigned URLs for academic submissions, alongside Amazon RDS PostgreSQL for relational modeling.
- **Observability & Security**: Real-time telemetry via Amazon CloudWatch, keyless credential handling via AWS Secrets Manager, and verified protection against unauthorized privilege escalation.

17.2 References

References

[1] IIITDM Kurnool, "Indian Institute of Information Technology Design and Manufacturing Kurnool Official Portal," 2026, accessed: Sep. 2026. [Online]. Available: https://www.iiitk.ac.in/

[2] Amazon Web Services, "Amazon Elastic Compute Cloud (EC2) User Guide," 2026, accessed: Sep. 2026. [Online]. Available: https://docs.aws.amazon.com/ec2/

[3] ——, "Amazon Simple Storage Service (S3) Documentation," 2026, accessed: Sep. 2026. [Online]. Available: https://docs.aws.amazon.com/s3/

[4] E. You and the Vite Team, "Vite: Next Generation Frontend Tooling Documentation," 2026, accessed: Sep. 2026. [Online]. Available: https://vitejs.dev/guide/

[5] Prisma, "Prisma Next-generation ORM for Node.js and TypeScript Documentation," 2026, accessed: Sep. 2026. [Online]. Available: https://www.prisma.io/docs/

[6] Amazon Web Services, "Amazon Relational Database Service (RDS) for PostgreSQL User Guide," 2026, accessed: Sep. 2026. [Online]. Available: https://docs.aws.amazon.com/rds/

[7] ——, "Amazon Cognito Developer Guide," 2026, accessed: Sep. 2026. [Online]. Available: https://docs.aws.amazon.com/cognito/

[8] N. Sakimura, J. Bradley, and N. Agarwal, "Proof Key for Code Exchange by OAuth Public Clients (RFC 7636)," Internet Engineering Task Force (IETF), 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7636

[9] Amazon Web Services, "Amazon API Gateway Developer Guide," 2026, accessed: Sep. 2026. [Online]. Available: https://docs.aws.amazon.com/apigateway/

[10] ——, "AWS Lambda Developer Guide," 2026, accessed: Sep. 2026. [Online]. Available: https://docs.aws.amazon.com/lambda/

[11] ——, "Amazon CloudWatch User Guide," 2026, accessed: Sep. 2026. [Online]. Available: https://docs.aws.amazon.com/cloudwatch/